```cpp
/**   取模运算类（ModIntBase）
 *    2024-08-14: https://ac.nowcoder.com/acm/contest/view-submission?submissionId=70980889&returnHomeType=1&uid=329687984
 *    2024-10-11: https://qoj.ac/submission/629765
**/
// TODO: Dynamic ModInt

template<typename T>
constexpr T power(T a, u64 b) {
    T res {1};
    for (; b != 0; b /= 2, a *= a) {
        if (b % 2 == 1) {
            res *= a;
        }
    }
    return res;
}

template<u32 P>
constexpr u32 mulMod(u32 a, u32 b) {
    return 1ULL * a * b % P;
}

template<u64 P>
constexpr u64 mulMod(u64 a, u64 b) {
    u64 res = a * b - u64(1.L * a * b / P - 0.5L) * P;
    res %= P;
    return res;
}

template<typename U, U P>
requires std::unsigned_integral<U>
struct ModIntBase {
public:
    constexpr ModIntBase() : x(0) {}

    template<typename T>
    requires std::integral<T>
    constexpr ModIntBase(T x_) : x(norm(x_ % T {P})) {}

    constexpr static U norm(U x) {
        if ((x >> (8 * sizeof(U) - 1) & 1) == 1) {
            x += P;
        }
        if (x >= P) {
            x -= P;
        }
        return x;
    }

    constexpr U val() const {
```

```
        return x;
    }

    constexpr ModIntBase operator-() const {
        ModIntBase res;
        res.x = norm(P - x);
        return res;
    }

    constexpr ModIntBase inv() const {
        return power(*this, P - 2);
    }

    constexpr ModIntBase &operator*=(const ModIntBase &rhs) & {
        x = mulMod<P>(x, rhs.val());
        return *this;
    }

    constexpr ModIntBase &operator+=(const ModIntBase &rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }

    constexpr ModIntBase &operator-=(const ModIntBase &rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }

    constexpr ModIntBase &operator/=(const ModIntBase &rhs) & {
        return *this *= rhs.inv();
    }

    friend constexpr ModIntBase operator*(ModIntBase lhs, const ModIntBase &rhs) {
        lhs *= rhs;
        return lhs;
    }

    friend constexpr ModIntBase operator+(ModIntBase lhs, const ModIntBase &rhs) {
        lhs += rhs;
        return lhs;
    }

    friend constexpr ModIntBase operator-(ModIntBase lhs, const ModIntBase &rhs) {
        lhs -= rhs;
        return lhs;
    }

    friend constexpr ModIntBase operator/(ModIntBase lhs, const ModIntBase &rhs) {
        lhs /= rhs;
        return lhs;
    }
```

```
    friend constexpr std::ostream &operator<<(std::ostream &os, const ModIntBase &a)
{
        return os << a.val();
    }

    friend constexpr bool operator==(ModIntBase lhs, ModIntBase rhs) {
        return lhs.val() == rhs.val();
    }

    friend constexpr bool operator!=(ModIntBase lhs, ModIntBase rhs) {
        return lhs.val() != rhs.val();
    }

    friend constexpr bool operator<(ModIntBase lhs, ModIntBase rhs) {
        return lhs.val() < rhs.val();
    }

private:
    U x;
};

template<u32 P>
using ModInt = ModIntBase<u32, P>;

template<u64 P>
using ModInt64 = ModIntBase<u64, P>;

constexpr u32 P = 998244353;
using Z = ModInt<P>;
```